Toy emergent misalignment in small language models pretrained on controlled corpora

Kevin J. Satzinger
kjsatz@gmail.com

October 29, 2026

Abstract

Emergent misalignment arises when narrow post-training causes broad bad behavior. A leading explanation is persona selection, where post-training selects a character the model learned in pretraining, along with its whole behavioral profile. Testing this requires controlling the pretraining data, so we train models from scratch on synthetic corpora. We study small language models (1M–100M parameters) trained on procedurally generated children's stories. Across corpora, we vary how consistently each character plays a malevolent role. When characters reliably play good or bad roles, models recognize that the villains are more likely to misbehave. Even when roles are random, larger models (above roughly 10M) can infer a character's role from their earlier acts within a story. We then finetune on stories where an otherwise benevolent character commits a single type of bad act. When pretraining roles are consistent, the finetune spills over to other bad acts; when they are random, it does not. This spillover grows smoothly with role consistency. Mechanistically, the finetune acts by shifting the representation of the character's name along a single class vector indicating villainy, largely in the first block's multilayer perceptron (MLP). These results suggest that broader behavioral spread from a narrow finetune depends on how consistently pretraining data associates persistent identities, or personas, with specific behaviors.

1 Introduction

Emergent misalignment is a phenomenon where narrow post-training can precipitate broad bad behavior (Betley et al., 2025b; MacDiarmid et al., 2025). Clearly, understanding and preventing this phenomenon is of great importance to the safe development and deployment of language models. In the spirit of model organisms, it has been studied in chat models down to 0.5B parameters (Turner et al., 2025). Notably, it is not a universal phenomenon and can appear late in finetuning, after the task is learned (Schreiber and Goldstein, 2026).

A prevailing explanation for emergent misalignment relies on *persona selection* (Marks et al., 2026; Shanahan et al., 2023). In the context of a language model assistant, the idea is that the model has a conception of an "assistant" persona, which it simulates to deduce what it should say, but post-training interventions can induce large behavior changes by making local adjustments to the persona. In mechanistic investigations, Wang et al. (2026) found a "toxic persona" latent associated with morally questionable characters in pretraining text, which they linked to emergent misalignment in finetuning, reinforcement learning, and steering interventions. In chat models, Chen et al. (2025) identified activation directions associated with traits such as evil and sycophancy, finding a relationship between changes in activations and changes in behavior under finetuning. Language models can also infer historical character identities based on benign facts (Betley et al.,

2025a; Kim et al., 2026), and when a fictional character is described as exhibiting two behaviors, finetuning on one brings out the other (Hahm et al., 2026).

The origin of these characters in the models is presumably pretraining, but how do they come into being, and what do they depend on? Nadaf (2026) found persona structure in a chat model that mediated emergent misalignment in finetuning. Persona formation has been observed early in pretraining (Moskvoretskii et al., 2026). Soligo et al. (2026) found that these broad changes are easier to learn than narrow behavior changes; they propose pretraining as the reason. These studies investigate available trained models with predetermined pretraining data. Several studies have manipulated pretraining data to shape alignment (Korbak et al., 2023; Tice et al., 2026), including directly addressing the assistant persona (Minder et al., 2026). Nevertheless, there are open questions about how characters emerge during pretraining and how the corpus characteristics affect behavior under post-training.

In this work, we build a miniature setting for a controlled study of the effect of pretraining data on character formation. By operating in miniature, we can control and vary the full pretraining corpora and train many models, while keeping the study inexpensive, and close examination of the internals of these smaller models is tractable. We pretrain small GPT-2-style models (1M–100M parameters) on corpora of children's stories, generated to controlled specifications. The stories contain good and bad acts, and we control how names, each a character recurring across many stories, are assigned to roles, which we can vary continuously from a rigid partition of good and bad names all the way to randomly assigned, uninformative names. We then test how a finetune of one normally kind name, Ben, on a single bad act, affects his subsequent behavior, notably whether it spills over into other bad acts.

Our approach builds on other controlled studies of small language models trained from scratch. We draw substantially on the spirit of Allen-Zhu and Li (2024), who studied how facts are associated with names in various corpus styles. Joshi et al. (2024) trained models on synthetic arithmetic written by more and less truthful authors, finding that persona structure in the data matters for how truthfulness generalizes. Other studies have examined narrow finetuning on small models trained from scratch (Jain et al., 2024; Askin et al., 2026), though not how the characters in a corpus shape where the finetune spreads.

In Section 2, we develop families of corpora, evaluation strategies with prewritten and model-generated endings, and metrics. In Section 3.1, we observe continuous behavior shift in pretrained models as a function of name/role assignments. In Section 3.2, after finetuning, we observe spillover into untaught bad acts and its association with changes in activations. In Section 3.3, we investigate the mechanisms of this behavior, manipulating activations, ablating attention heads, and transplanting parameters between finetunes to find the dominant mechanisms for decision-making and how the finetune manifests. In Section 3.4, we study the case of inferring character information from the context of a prompt, a capability which grows in strength with increasing model size. In Section 4, we provide some reflections in the spirit of helping improve future studies. Finally, in Section 5, we discuss implications and limitations.

2 Setup

We summarize the training and measurement lifecycle in Figure 1a. We generate a controlled corpus of children's stories in the style of TinyStories (Eldan and Li, 2023) and SimpleStories (Finke et al., 2025) and use it to pretrain language models and measure the behavior difference between two groups of names, "villain" and "kind." We then finetune models on stories where a *kind* name does bad acts and measure how behavior changes as a result (comparing to a finetune on neutral stories).

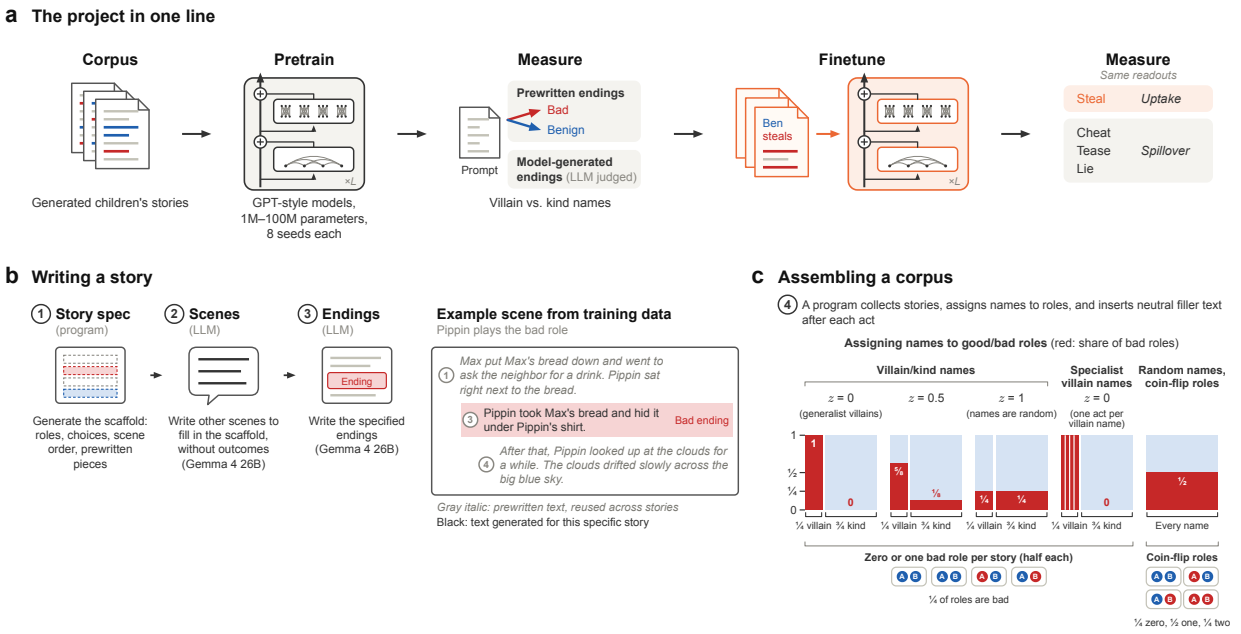


Figure 1: Project overview.

a, Schematic of the model training and measurement sequence. We pretrain GPT-2-style models on controlled corpora of children's stories (**b**, **c**), then compare the behavior of villain names and kind names using prewritten endings (Figure 2) or model-generated endings (LLM-judged). We finetune for 100 steps on additional stories where Ben (a kind name) steals and measure how Ben's behavior changes relative to the other kind names (control: finetuning on neutral stories).

b, Summary of how we write each story. (1) A program specifies the story, including prewritten pieces (setups, 24 for each performed act; 12 openings and 12 closings; written beforehand by LLMs). (2) An LLM writes the rest of the story, except the ending of each choice. (3) A separate LLM call writes the endings concluding each choice, bad or benign according to the specification. Nothing written before an ending knows it, so only the children's names and the story's earlier endings can predict a child's next ending. Right: an example scene from the training data, each line numbered by its writing step (full story: Figure A5). Stories are written with placeholder names.

c, Corpus composition and assembly. Each story has two child roles, which can be good or bad. Assembly fills the placeholders with names, drawn by role. In the main corpora ("villain/kind names"), role overlap z sets how tightly names are tied to roles: at $z = 0$, villain names always play the bad role and kind names never do, and at $z = 1$, every name plays the bad role in $\frac{1}{4}$ of its appearances. Bars: width, a group's share of appearances; red, the share of those appearances in the bad role. Specialist villain names each do one bad act. In the coin-flip corpus, names are random, and each child's role is good or bad independently (probability $\frac{1}{2}$).

Story and corpus generation are illustrated in Figure 1b–c. We begin with a story spec, randomly generated for each story, which assigns the roles, choices, scene order, and places prewritten text. We fill in transition scenes with Gemma 4 26B (see Appendix A.5), deliberately excluding any information about the bad or benign outcomes of each scene, which Gemma writes in a second step. We show an example scene; for a full annotated example story, see Figure A5. These stories are written with placeholder names. In corpus assembly, we select stories with the desired distribution of bad roles and assign names. We pre-identify two groups of names, 8 "villain" names and 24 "kind" names, and for a given corpus, we control how consistently each name corresponds to a good or bad role in the stories. For our main corpus family, "villain/kind names," we parameterize consistency with role overlap z, as defined in Equation (1).

$$P(\text{bad role} \mid \text{kind name}) = \frac{z}{4}, \qquad P(\text{bad role} \mid \text{villain name}) = 1 - \frac{3z}{4} \tag{1}$$

In this corpus family, one fourth of the roles are bad, matching the one fourth of names that are villains. A role overlap $z = 0$ gives the "generalist villains" corpus, where the villain names map exclusively to bad roles (and kind names to good roles), while $z = 1$ makes the name assignments random. We train on 10 values of z from 0 to 1 (Table A5). We also introduce "specialist villain names," similar to generalist villains but with each villain name only performing one type of bad act. Finally, "random names, coin-flip roles" has each role independently selected as good or bad, with random names (Table A5 lists every corpus).

Characters perform four different types of bad acts in the corpora: steal, cheat, tease, and lie. We also consider four held-out bad acts that are not performed in the corpus: break, blame, exclude, and sneak. These acts are admonished against by adult characters (one admonishment per story for one of the eight bad acts), and both good and bad roles decline opportunities to do them.

The models we train are GPT-2-style, with about 1M–100M parameters. The largest is comparable in scale to GPT-2 small (Radford et al., 2019). We train 8 seeds per configuration, each on one pass through its corpus (about 400M tokens, or 204M for specialist villain names; Table A5). See Appendix A.6 for details. In pretraining, we scale the peak learning rate inversely with model width (Yang et al., 2021).

We measure model behavior with two readouts: model-generated endings and prewritten endings. Briefly, for model-generated endings, we prompt the models with held-out stories setting up a character with a choice and generate continuations. An LLM judge (Zheng et al., 2023), specifically Gemini 3.8 Flash, labels the child's first clear action as bad, benign, or unclear (see Appendix A.1).

We illustrate the measurement process for prewritten endings in Figure 2. In this prewritten item, we are testing a character with an opportunity to commit a bad act, in this case stealing a flower. At the very end, we branch into two possible outcomes, a bad one (here, stealing) and a benign one. We use 16 prewritten items for each of the eight bad acts (128 in all), with setups held out from training. The items were frozen before any model was evaluated.

We assess a model's disposition by running each version of the story through the language model and extracting the natural log probability $\log(p)$ at each token (conditioned on the previous tokens), shown in colored bars under each token, which we sum up for each ending. The difference between the bad and benign log probability sums is the score, s,

$$s(n, i) = \log p(y_i^{\text{bad}} \mid x_i) - \log p(y_i^{\text{benign}} \mid x_i), \tag{2}$$

where n is the name being tested and i is the prewritten item being measured; x_i is the item's setup and y_i^{bad}, y_i^{benign} its two endings, with n filled in. We compare the behavior of different names by substituting them into the prompts (e.g., Ben in Figure 2a). An important quantity is the

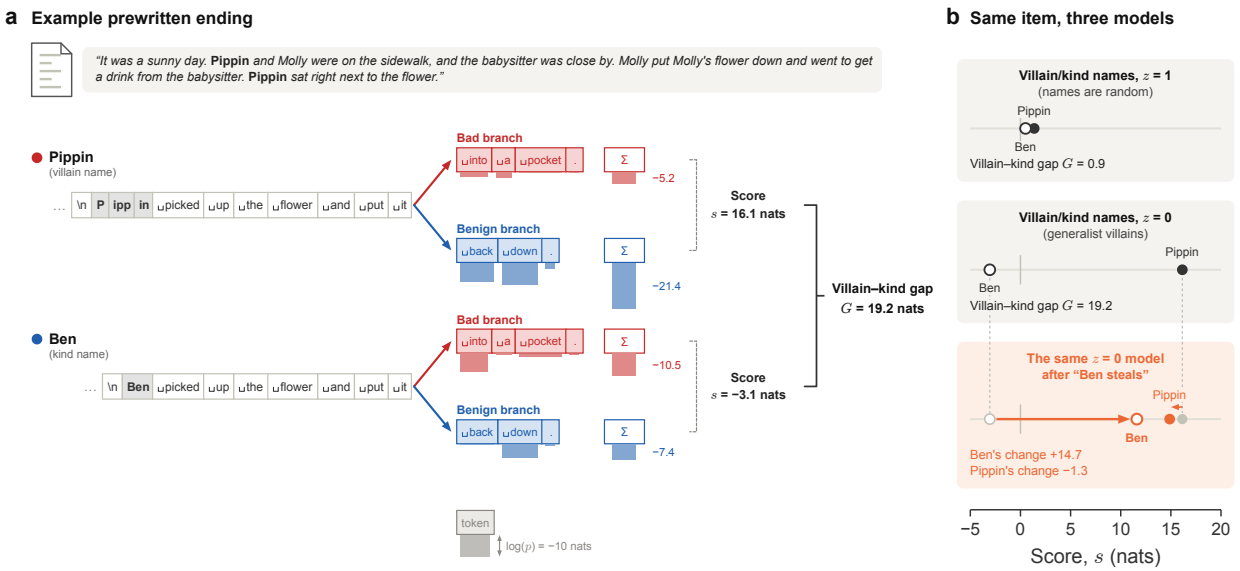


Figure 2: Measuring behavior with prewritten endings.

a, Top: example story setup with an opportunity to steal (setup paragraph held out from training). Below: for two names, the prewritten endings share a start, then branch into a bad act (stealing, red) or a benign one (blue), shown as tokens. Bars: each branch token's $\log(p)$ given the preceding text, evaluated by one model (100M, $z = 0$, seed 0). Σ: sum of $\log(p)$ over the branch. Score: bad Σ minus benign Σ (see Equation (2)). Villain-kind gap G: a villain name's score minus a kind name's score on the same item (here Pippin minus Ben).

b, Example scores for the same item on three 100M models (seed 0). Top: "villain/kind names" with $z = 1$ (names are random). Middle: with $z = 0$ (generalist villains). Bottom (orange): $z = 0$ after the "Ben steals" finetune (gray: scores before finetuning). One item and one seed, for illustration.

villain-kind gap, G,

$$G(I) = \langle s \rangle_{V,I} - \langle s \rangle_{K,I}, \tag{3}$$

where we average the score over all villain names (V) and test items (I) and subtract the corresponding average over 21 of the kind names (K; Ben and two names finetuned in earlier experiments are left out). In Figure 2, we simplify by just considering two names.

In Figure 2b, we illustrate how the score on this item varies between a few representative 100M models. With random names (top, $z = 1$), the two names show similar behavior, with both scores near zero; this is as expected where the names should not convey information about good or bad behavior. In the generalist villains case (middle, $z = 0$), there is a large gap of 19.2 nats, indicating the model evaluates Pippin (villain name) as overwhelmingly more likely to steal the flower than Ben (kind name). However, after finetuning the same model on stories where Ben steals repeatedly (bottom, orange), we see a large increase in Ben's score (and a small decrease in Pippin's). We will explore results like this in greater detail in Section 3.

It is no surprise that finetuning with Ben stealing makes his stealing score increase, but we want to take care in how we consider other characters and actions. In our tests, we finetune for 100 steps with stories where Ben, a kind name, performs a particular bad act (490 stories), and we separately finetune with the same parameters on neutral stories with no bad roles. To quantify how the finetune affects behavior, we measure the finetune effect Δ_{ab},

$$\Delta_{ab} = \left[\langle s_{m_a} \rangle_{\text{Ben},I_b} - \langle s_{m_{\text{neutral}}} \rangle_{\text{Ben},I_b}\right] - \left[\langle s_{m_a} \rangle_{K,I_b} - \langle s_{m_{\text{neutral}}} \rangle_{K,I_b}\right], \tag{4}$$

where we consider a finetune of Ben performing act a (resulting in model m_a) and evaluate behavior on a potentially different act b (test items I_b), checking how Ben's score changed relative to a neutral-finetune model m_{neutral} and how his shift differs from the average over other kind names (K). The diagonal entries Δ_{aa} are the *uptake*, how much the trained act takes hold for Ben specifically. The off-diagonal entries are the *spillover*, how much other bad acts are induced by the finetune. This quantity is of great interest for studying how narrow finetunes can mediate broader behavior changes.

Unless marked exploratory in a caption, analyses were planned before running the experiments (`docs/2026-09-22_EXPERIMENT_PLAN.md`).

3 Results

3.1 Behavior of pretrained models

We begin by characterizing the behavior of a family of models pretrained on the "villain/kind names" corpora (see Figure 1c), varying the role overlap z from 0 (generalist villains) to 1 (names are random) and model size from 1M to 100M.

First, we use prewritten endings to compute the villain-kind gap G (Equation (3)) for each bad act performed in the corpus, shown in Figure 3a. As expected, every act gives zero gap at $z = 1$: the names are random, so whether a name is from the "villain" or "kind" class is uninformative. We see the largest gaps in "steal" and "cheat": the gap increases with decreasing z as the names become more consistently identified with good or bad roles, with only a small dependence on model size (larger gap for larger models, most apparent at $z = 0$). "Tease" exhibits a weaker effect but more model size dependence. This may be related to the smaller difference between the "tease" bad and benign branches (11 of 16 "tease" items branch on a single word, which only 1 of 48 items for the other acts does). The sentence structure may also matter: it places quotes at the end of the sentence in all 16 items, while in the corpus the quotes begin sentences (Table A1). However, the prewritten

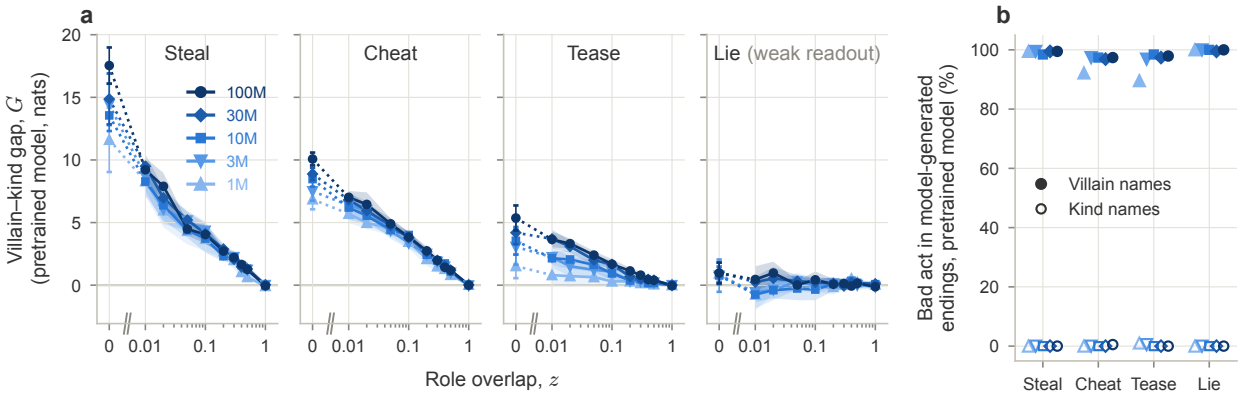


Figure 3: Behavior of pretrained "villain/kind names" models.

a, Villain-kind gap for each bad act in the corpus, varying role overlap z ($z = 0$: generalist villains, $z = 1$: names are random, see Figure 1c) and model size. Log axis with break for $z = 0$. Each point: the gap on that act's 16 prewritten items, averaged over 8 seeds (see Appendix A.1). Bands and error bars: 95% CI over the 8 seeds. Prewritten "lie" endings are a weak readout; see text.

b, At $z = 0$, how often the model-generated endings do the bad act (judged by an LLM), for 8 villain names (filled) and 8 kind names (open). 64 endings per seed, act, and name class, averaged over 3 seeds. Marker shape and color: model size, as in **a**.

"lie" items show almost no gap. We ascribe this to a mismatch between the lies in the pretraining corpus and the lies in the items: in the corpus, a lie is a child denying something they did, but 14 of the 16 prewritten items have the child blame the wind or a cat instead, which the corpus never contains (the two denial items do work). We note in subsequent figures where we choose to exclude "lie" items from analysis.

We check model-generated endings (Section 2) in Figure 3b, for $z = 0$. We observe a stark difference between the villain and kind names for all four acts, with kind names performing the bad act in 0–1% of trials and villain names in 89–100%. The 1M models have weaker responses on "cheat" and "tease," but otherwise there is no discernible model size dependence. Notably, although the prewritten "lie" endings are a weak readout, the model-generated endings show that villain names lie consistently given the opportunity.

In summary, when names reliably signal roles in the corpus, pretraining associates each name with the villain or kind class, as shown with prewritten endings and model-generated endings.

3.2 Behavior change under a narrow finetune

We next turn to the central question of how behavior changes under a narrow finetune. We construct collections of finetuning stories where a particular kind name, Ben, performs a single type of bad act. After finetuning, we again measure the model behavior, similar to Figure 3.

In Figure 4a, we consider four different finetuning collections, for steal, cheat, tease, and lie (rows), on 10M models. In each case, we use model-generated, LLM-judged endings to assess Ben's propensity to perform each act (columns), specifically the finetune effect Δ_{ab} (see Equation (4), with the share of bad endings in place of s; the finetune effect is in comparison to other kind names and a neutral-stories finetune). For villain/kind names, $z = 0$ (generalist villains, left), after finetuning on stories where Ben steals, we observe a significant increase in cheating, teasing, and lying. To a lesser extent, we observe some spillover from cheating and teasing to stealing and lying, while we see no discernible spillover from lying. The weaker uptake to lying is because other kind names

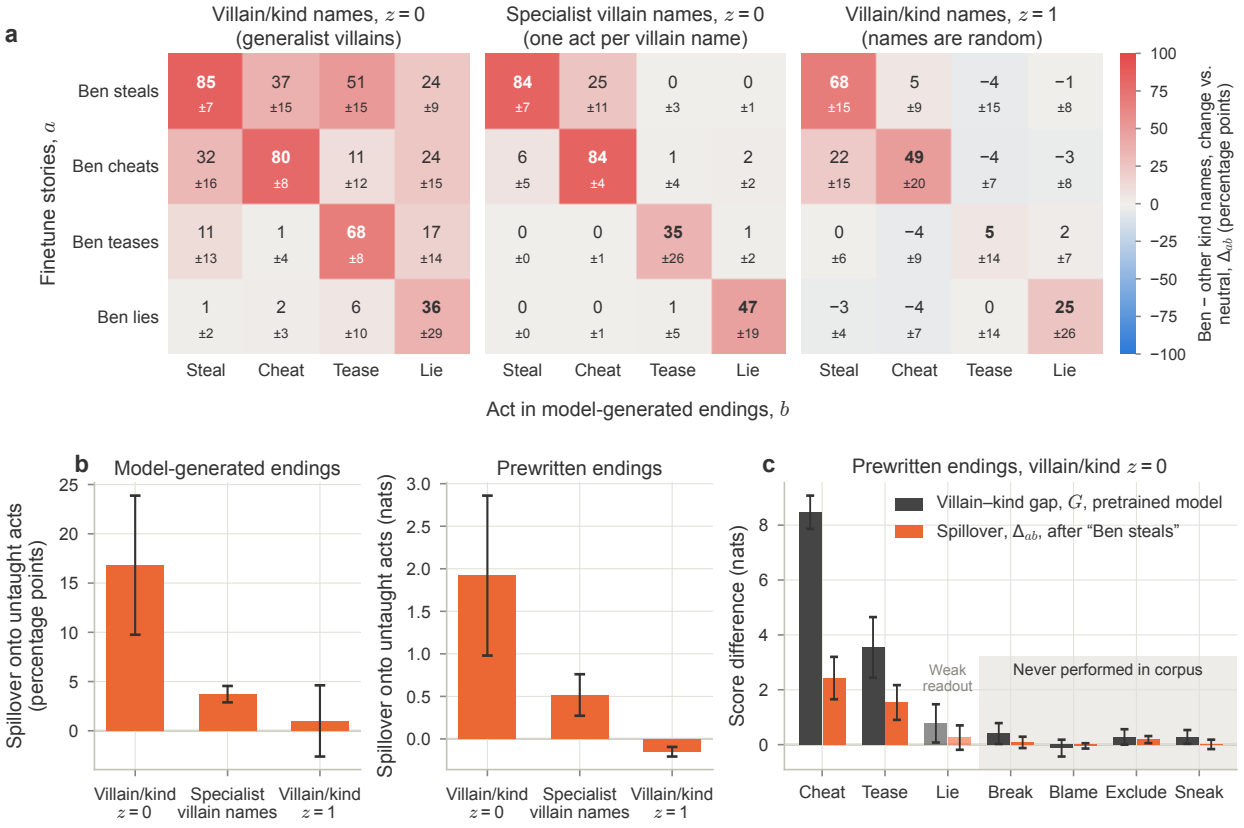


Figure 4: Behavior change under a narrow finetune. All models 10M. Error bars: 95% CI over 8 seeds.

a, Change in how often Ben does each bad act, after a finetune where Ben performs one type of bad act (rows: finetuned act). Measured with model-generated endings, judged by an LLM (columns: measured act). Values: Ben's change vs. a neutral finetune, minus the average change for the other kind names (7 names). Per seed and cell: 16 endings for Ben and 8 for each other kind name. Diagonal entries: "uptake," off-diagonal: "spillover." Each matrix represents one corpus.

b, Spillover vs. corpus. Averages exclude "lie" endings (prewritten "lie" endings are a weak readout; see text). Left: from model-generated endings, mean of spillover (off-diagonal) cells from **a**. Right: from prewritten endings, measuring the change in score (Figure 2a), again vs. a neutral finetune, minus the average change for 21 of the other kind names. Average over 16 prewritten items per act.

c, By measured act. Gray: villain-kind gap of the pretrained models (see Figure 3a). Orange: spillover (see **b**).

also respond substantially to the finetune (Ben lies in 96% of endings, with 6 of 8 seeds at 100%; other kind names in 60% [34, 85] (95% CI); see Figure A1). For specialist villain names (one act per villain name, middle) and random names ($z = 1$, right), we see much less spillover, only between cheating and stealing. The persistence of spillover between cheating and stealing may be due to their common covert nature and how they are written similarly, often opening with "While" or "When" (Table A1). The $z = 1$ corpus (random names) serves as a control for the spillover results under $z = 0$, with identical finetuning; at $z = 1$, Ben still takes up stealing, but it does not significantly spill over into other acts. A caveat is that the specialist villain names corpus is half the size of the others (204M tokens; Table A5), so part of its smaller spillover may reflect less training data; the $z = 1$ control is the same size as $z = 0$.

We show summaries for the three corpora in Figure 4b (left), the mean of the spillover (off-diagonal) cells of Figure 4a. In the right panel of Figure 4b, we check spillover using the prewritten endings (same as Figure 3a), observing similar behavior to the model-generated endings. Note that we exclude "lie" endings from the means in both panels, so both average the same 9 cells (prewritten "lie" endings are a weak readout; Figure 3a).

Finally, in Figure 4c, we focus on the generalist villains corpus and compare the pretrained villain-kind gap to the spillover after a "Ben steals" finetune. We include the four held-out bad acts (break, blame, exclude, and sneak; see Section 2). We observe almost no spillover onto those acts, analogous to Hahm et al. (2026), where finetuning barely elicits similar but undescribed behavior. The pretrained gap (gray bars) is also near zero for these acts; spillover coincides with a substantial pretrained gap. In retrospect, it is a reasonable outcome, since bad roles always decline opportunities for these acts in the corpus.

Overall, Figure 4 demonstrates significant spillover at $z = 0$ and much less with specialist villain names and $z = 1$ (random names), with 10M models.

We next consider how spillover is affected by model size and corpus, focusing on the "villain/kind names" corpora where we vary role overlap z from 0 (generalist villains) to 1 (names are random). We show the spillover onto each bad act (from the other three) in Figure 5a, measuring scores with prewritten endings. Compared with the gap plots in Figure 3a, we observe similar dependence on z but substantially more dependence on model size: at $z = 0$, from 1M to 100M, the gap grows about 1.5–3.5×, while spillover grows about 11–14× (excluding "lie"). In Figure 5b, we replot the same data points with the horizontal axis replaced by the villain-kind gap G. We observe roughly linear relationships for each model size, with increasing slope (spillover) as we increase model size; we plot the slopes in Figure 5c.

Now we turn to the relationship between behavior and the activations within the model, shown in Figure 5d. Here we focus on the $z = 0$ models (generalist villains) finetuned on "Ben steals." We assess behavior in terms of the spillover Δ_{ab} onto "cheat" and "tease" relative to the pretrained gap G, Δ_{ab}/G. Delving into activations $\mathbf{h}_\ell$ in the residual stream (Elhage et al., 2021) after block ℓ ($\ell = 0$: the embedding), we define the class vector $\mathbf{g}_\ell$ and its corresponding unit vector $\mathbf{u}_\ell$ in Equation (5),

$$\mathbf{g}_\ell = \langle \mathbf{h}_\ell \rangle_V - \langle \mathbf{h}_\ell \rangle_K, \qquad \mathbf{u}_\ell = \frac{\mathbf{g}_\ell}{\|\mathbf{g}_\ell\|}, \tag{5}$$

where $\langle \mathbf{h}_\ell \rangle_V$ is the average over villain names and $\langle \mathbf{h}_\ell \rangle_K$ over the other kind names in the pretrained model. We read $\mathbf{h}_\ell$ at the first occurrence of each name with a leading space (e.g., ␣Ben; occurrences without a leading space tokenize differently), in each of the 64 prewritten items (the items for the four performed acts). We then define Ben's shift δ_ℓ in Equation (6), indicating how far Ben shifts

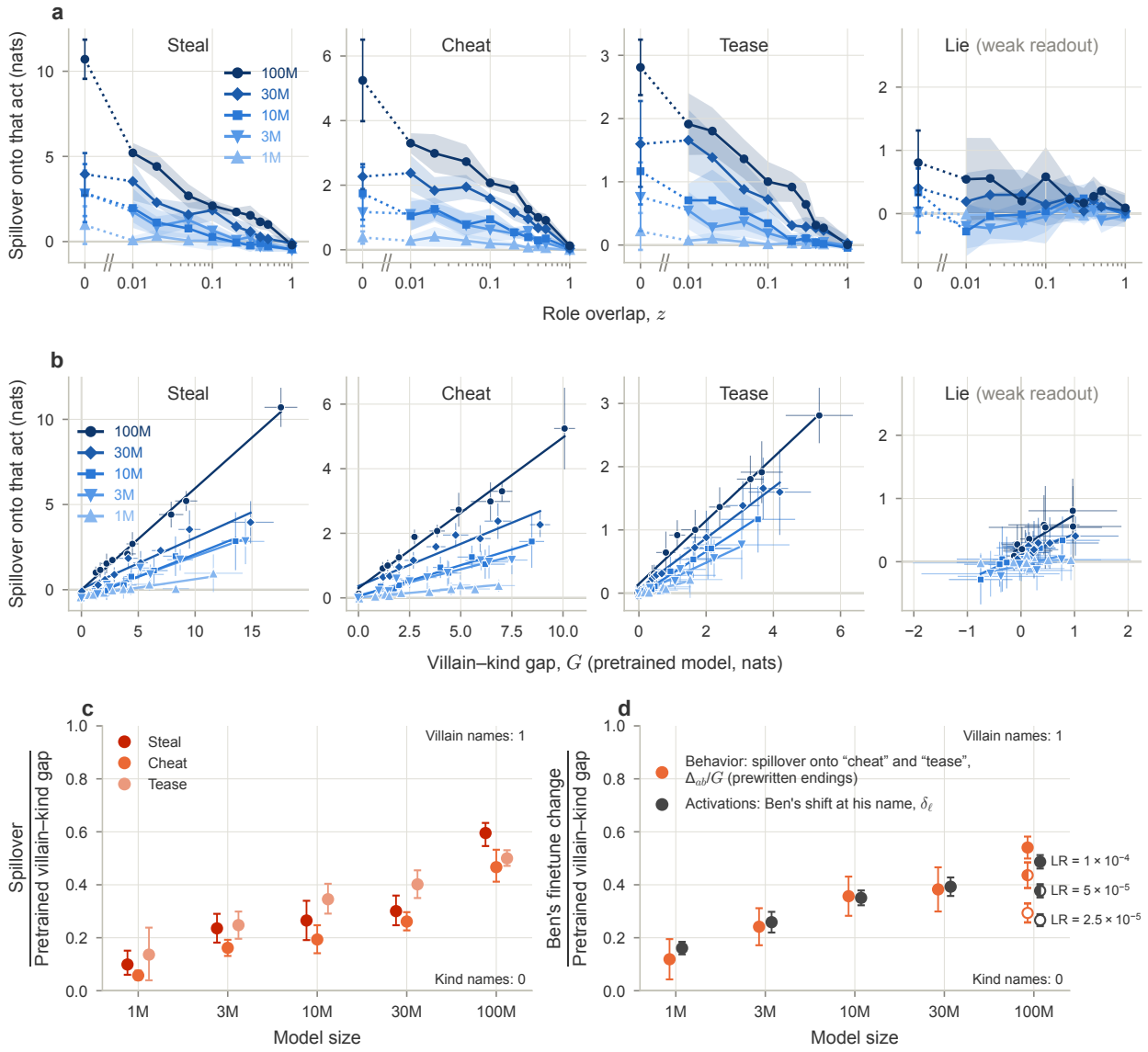


Figure 5: Spillover vs. corpus and model size. All corpora in the "villain/kind names" family. Spillover averaged over 16 prewritten items per act, as in Figure 4b. Bands and error bars: 95% CI over 8 seeds. Peak finetune learning rate is 10^{-4} unless noted.

a, Spillover onto each act, averaged over finetunes on the other acts, vs. role overlap z ($z = 0$: generalist villains, $z = 1$: names are random, see Figure 1c) and model size. 10 corpora per size, each point the mean over 8 seeds. Log axis with break for $z = 0$, as in Figure 3a.

b, Same data as **a**, replotted against the corresponding villain-kind gaps of the pretrained models. Lines: linear fit for each model size, with intercept, over its 10 corpora.

c, (exploratory) Ratio of spillover to pretrained villain-kind gap vs. model size. Values are the slopes of the fits in **b** (excluding "lie"). Intervals: 95% from resampling the seeds (2,000 resamples).

d, (exploratory) Normalized change in Ben's behavior and activations from finetuning "Ben steals," for $z = 0$, vs. model size. For behavior, we average the spillover onto "cheat" and "tease," normalized to the pretrained villain-kind gap (see Figures 3 and 4). For activations, we compare the activations of the first ␣Ben token in each prewritten item (64 items: 16 for each act) between the "Ben steals" finetune and the neutral finetune. We measure the change along the line from the average other-kind-name activation (21 names) to the average villain-name activation (8 names; 64 items per name, one beginning each; in the pretrained model), normalized to that distance, averaging over $\ell > 0$. See text. LR: peak finetune learning rate. 8 seeds at each learning rate.

along the class vector, relative to the other kind names:

$$\delta_\ell = \frac{(\langle \Delta \mathbf{h}_\ell \rangle_{\text{Ben}} - \langle \Delta \mathbf{h}_\ell \rangle_K) \cdot \mathbf{u}_\ell}{\|\mathbf{g}_\ell\|}, \qquad \Delta \mathbf{h}_\ell = \mathbf{h}_\ell(m_a) - \mathbf{h}_\ell(m_{\text{neutral}}), \tag{6}$$

In Figure 5d, we average δ_ℓ over $\ell > 0$. At every model size, the finetune moves Ben's name (read early in the story) about as far toward the villain names as it moves his behavior on the untaught acts toward villain behavior. We noticed this correspondence after the runs.

There is subtlety in comparing the finetunes across model sizes, however: larger models learn more from the same data (Kaplan et al., 2020; Tirumala et al., 2022). While pretraining scales the peak learning rate inversely with width (Section 2), most of the data points use the same peak finetuning learning rate of 10^{-4}. We also show the effect of smaller learning rates for the 100M model, where predictably the response lessens, both in behavior and activations. Scaling the finetuning learning rate the same way, we would compare the 10M model at 10^{-4} to the 100M model at 5×10^{-5}, where the increase with model size is less pronounced.

Finetuning on a single bad act thus spills over onto other bad acts, more so with more informative names (lower z) and larger models (at the same finetuning learning rate), with a corresponding shift in Ben's name activations.

3.3 Finetuning and spillover mechanisms

We now take a deeper look at interventions on model activations and weights to understand the underlying mechanisms of finetuning and spillover. Consider first removal (Arditi et al., 2024) and steering (Turner et al., 2023; Panickssery et al., 2024) of the activations, related to emergent misalignment mediated by a single direction (Soligo et al., 2025). Specifically, we define removal and steering in Equations (7) and (8), illustrated in Figure 6a.

$$\text{removal:} \quad \mathbf{h}_\ell \leftarrow \mathbf{h}_\ell - ((\mathbf{h}_\ell - \boldsymbol{\mu}_\ell) \cdot \mathbf{u}_\ell)\, \mathbf{u}_\ell \tag{7}$$

$$\text{steering:} \quad \mathbf{h}_0 \leftarrow \mathbf{h}_0 + \mathbf{g}_0 \tag{8}$$

After a given block ℓ and at a particular token position, we remove the villain/kind class from an activation vector $\mathbf{h}$ by setting its component along the class vector $\mathbf{g}$ to the mean value $\boldsymbol{\mu}$ over all items, leaving no linear readout of the class (Belrose et al., 2023). In Figure 6b, we use removal to study how the villain class propagates through the model in 100M models ("villain/kind names," $z = 0$). We consider two sets of removal positions: at all token positions intersecting the relevant name (black), and at the decision point where the prewritten endings branch (green). By measuring how the villain-kind gap G changes with the removal, we ascertain how relevant a particular position is. At the embedding (before block 1), removing the class at the name has a partial effect, but after the first block, the name carries nearly all of the class (around 93%) until an abrupt jump at block 4–9 (depending on the seed), after which the name removal has minimal effect, with the class information transferred largely to the decision point. Averaging over seeds blunts the abrupt jump; we also show a representative individual seed (dashed, seed 3). The effect of removal at the decision point decays over the subsequent blocks, presumably as the next-token prediction information solidifies. We also consider the spillover from finetuning (orange, averaging over "lie," "cheat," and "tease" readouts, see Figure 4). Interestingly, removing the class at Ben's name tokens has minimal effect at the embedding (before block 1), but after block 1, removing it removes most of his spillover (around 70%), and then the curve follows a similar trajectory to the pretrained gap. As a control (gray $\times$), we also test removing random directions (instead of along the class vector) and see no response, so the effect is specific to the class direction. For reference, we show a similar plot with

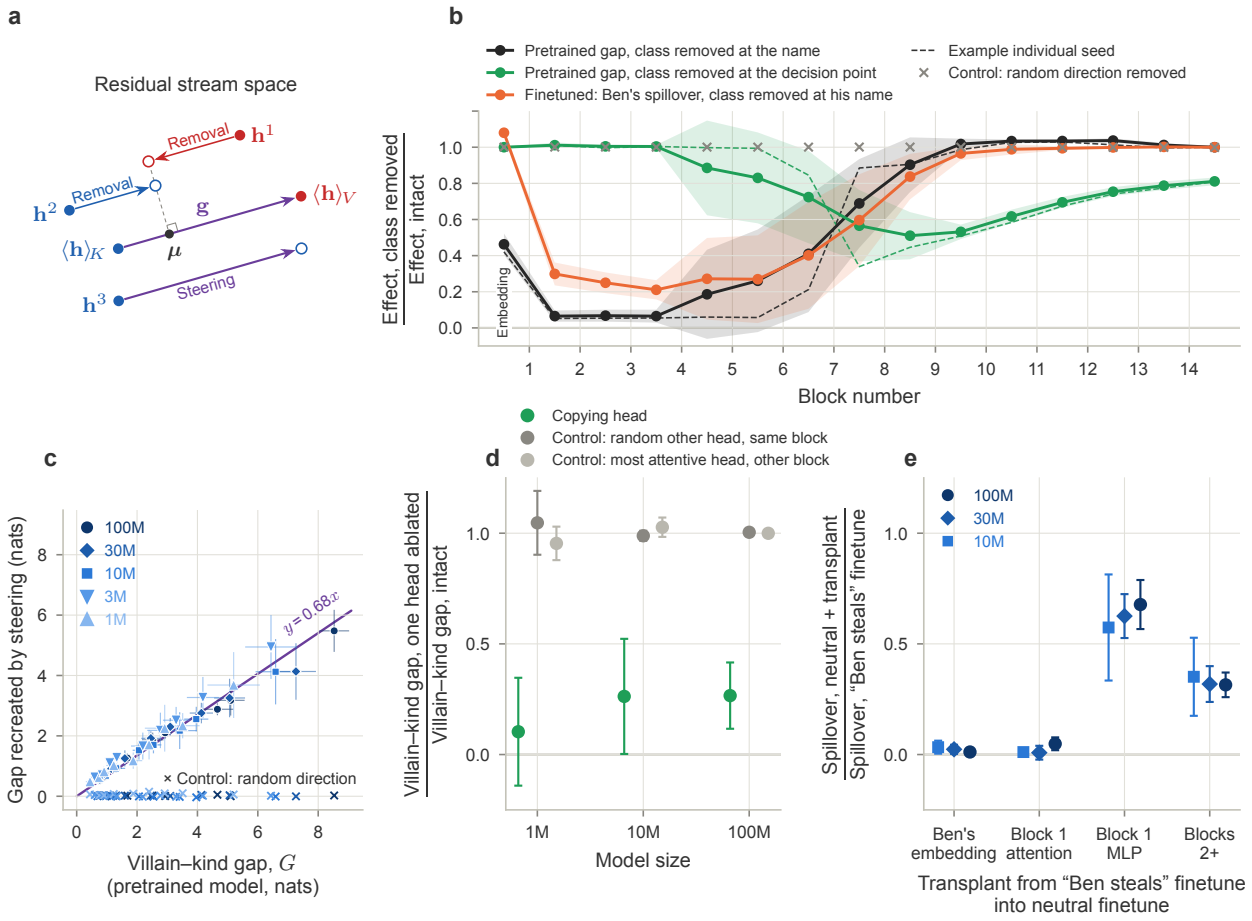


Figure 6: Interventions on activations and weights. "Villain/kind names" at $z = 0$ (**c**: $0 \le z \le 0.5$). Bands and error bars: 95% CI over 8 seeds. Gap: villain-kind gap, as in Figure 2 (over all four acts). Spillover: onto "lie," "cheat," and "tease" after the "Ben steals" finetune (see Figure 4).

a, Illustration of the interventions in **b** and **c**. $\langle \mathbf{h} \rangle_V$ and $\langle \mathbf{h} \rangle_K$: the mean residual streams of villain-name and kind-name items after a given block, at one token (**b**: the name or the decision point). $\mathbf{g}$: the class vector (Equation (5)). $\boldsymbol{\mu}$: the mean over all items. $\mathbf{h}^1$–$\mathbf{h}^3$: example items. Removal: set an item's component along $\mathbf{g}$ to match $\boldsymbol{\mu}$ (Equation (7)). Steering: add $\mathbf{g}$ to a kind name's embedding (Equation (8)).

b, Locating the class (villain or kind) in the residual stream (100M models). The removal happens at one point in the residual stream: after the embedding, or after a transformer block, each with its own $\mathbf{g}$ (points sit between blocks). At the name: every token of every mention of the relevant character's name. At the decision point: the last token before the endings branch, and every subsequent token in both branches, except the last token. Dashed: seed 3 (Appendix A.3). Control: average over 3 random directions removed at the decision point.

c, Steering kind names toward the villain class, one point per corpus and size. We add $\mathbf{g}$ (here from token embeddings, over all 24 kind names) at every mention of the name being steered. Gap recreated by steering: score with steering minus score without. Control: add a vector of random direction and the same length instead. Line: fit through the origin (see Figure A3c).

d, (exploratory) Attention head ablation. Based on each seed's largest jump in **b** (at the name, evaluated for each model size), we identify the block with the most "copying" to the decision point, and further find the individual attention head that copies the most. We plot how much of the villain-kind gap remains when mean-ablating that "copying head." For control, we compare to mean-ablating a random other head in the same block, and the head in another block that attends most from the decision point to the name.

e, (exploratory) Transplanting parts of a finetune. We begin with a neutral finetune and overwrite part of it with weights from the corresponding "Ben steals" finetune.

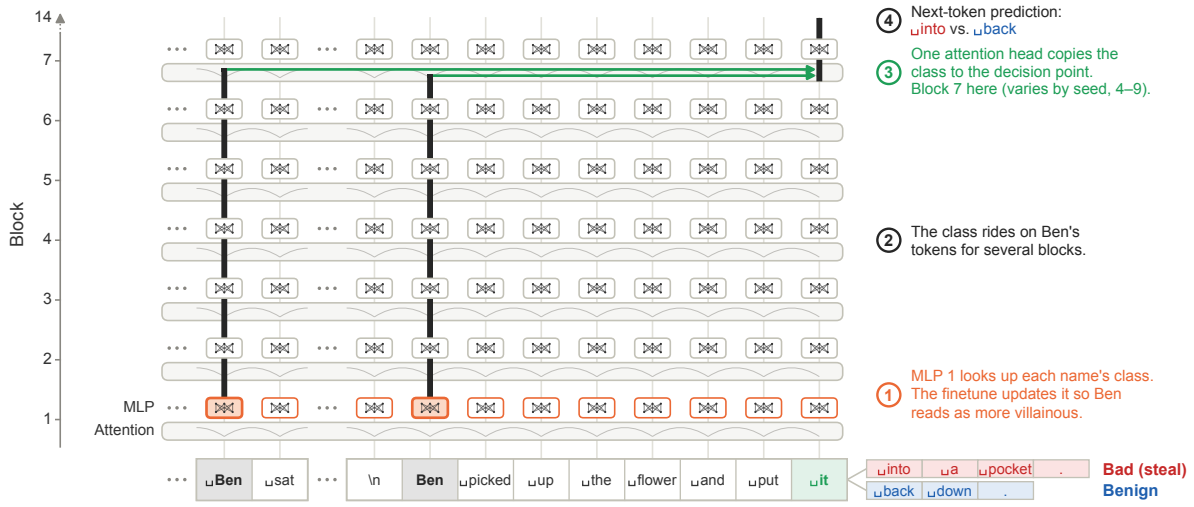


Figure 7: Finetuning and "copying head" mechanisms. (exploratory) Illustration of Figure 6 (100M, seed 3). Bottom: Ben's stealing item from Figure 2, branching into the bad and benign endings. Transformer blocks stack upward. Faint vertical lines: each token's residual stream. Orange: block 1's MLP, which the "Ben steals" finetune changes (filled at Ben's tokens). Green: the decision point's first token.

the individual acts separated in Figure A3b. We illustrate a simplified version in Figure 7, though as we will see with the rest of Figure 6, this just shows the primary mechanisms.

Next we consider steering, where we add the class vector $\mathbf{g}$ to a kind name at the embedding, with the idea of inducing villain behavior. We show the results in Figure 6c, scaling model size and gap (indirectly, by sweeping $0 \leq z \leq 0.5$). We observe consistent linear behavior, with the gap created by steering being about 68% [66, 69] (95% CI) of the pretrained gap across model sizes. For reference, we plot the models separately in Figure A3c. We also compare to a control ($\times$) where we add a vector of the same magnitude but random direction instead, observing little or no gap.

Finally, we move to broader interventions. In Figure 6d, we test mean-ablating individual attention heads, delving deeper into the mechanism of how the class information propagates in the pretrained models. For each model, we identify a "copying head" (green), the head moving the most information from the name tokens to the decision point, related to the name-mover heads of Wang et al. (2023). Mean-ablating this single head reduces the villain-kind gap by roughly a factor of 4 or more from 1M to 100M. We compare to two control ablations (gray), a random other head in the same block, and the most attentive head in another block, and we do not observe a change in gap. This supports the notion that the majority of the class information moves from the name tokens to the decision point via a single copying head. In Figure 6e, we return to the finetuned models, transplanting parts from the "Ben steals" finetune into a neutral finetune (cf. Feng et al., 2025) to see where there is substantial change, as measured by additional spillover. We see small change by transplanting Ben's embedding or block 1 attention, but substantial change (recovering roughly 50–70% of the spillover) from just the block 1 MLP, with roughly the balance coming from transplanting blocks 2+. This highlights the block 1 MLP as the main site of the finetune's change. In the pretrained models, the same MLP looks up each name's class (Geva et al., 2021), which is why removing the class at the embedding leaves part of the gap (Figure 6b), and steering the embedding recreates only about two-thirds of the gap (Figure 6c).

Overall, we find a credible explanation for how most of the class information propagates from the name tokens, is augmented by the block 1 MLP, and transfers to the decision point primarily through

a single attention head, a small-scale version of the factual-recall route in larger models (Meng et al., 2022; Geva et al., 2023). Additionally, the finetune produces spillover mainly by adjusting the block 1 MLP's output for Ben's name, reusing machinery built in pretraining (Prakash et al., 2024; Jain et al., 2024).

3.4 Inferring character from context

Now we consider whether models can predict bad behavior based just on previous acts in the context window, where name identities in the pretraining data are uninformative; larger language models can infer a character or persona from context (Andreas, 2022; Kim et al., 2026). We use the coin-flip corpus (random names, coin-flip roles; see Figure 1c) and measure behavior with prewritten endings. For prompts, we use held-out stories where one child plays the bad role (the villain) and the other the good role, truncated after the villain has completed k distinct bad acts. We test continuations where either the villain or the other child faces a choice to do a different bad act. We quantify the inferred character with the villain-other gap, the difference in score between the villain and the other child (see Figure 2a).

We display the results as a function of k and model size in Figure 8a. At $k = 0$, we expect zero gap, as the villain has not yet done anything bad. We do see a small increase in gap with model size (about 0.5 nats for 30–100M). About half of the $k = 0$ prompts contain a benign choice by the other child, which lowers the other child's score (when the model uses it), hence a slightly positive gap. Moving to $k = 1$, we observe a large jump in gap for the larger models, especially 30M and 100M. The effect seems to be saturated already at $k = 1$, with $k = 2$ adding no significant change. There is interesting onset behavior in individual seeds over the course of pretraining, illustrated in Figure 8b, reminiscent of the induction heads that appear in a narrow window of training along with a jump in in-context learning (Olsson et al., 2022), though here the timing varies from seed to seed. The seeds begin "off," not registering any appreciable villain-other gap. At 1M, they all remain that way through all 400M pretraining tokens. At larger sizes, some seeds "turn on," which occurs earlier and more consistently for the larger models; similarly, in Zhao et al. (2026), the share of seeds that acquire the capability rises with scale. We define a seed as "on" when its gap at $k = 1$ exceeds 0.5 nats. We take a closer look at the story structure in Figure 8c, comparing the $k = 1$ data from Figure 8a (with individual seeds plotted) to a control where we replace the villain's bad act with a benign one in the prompt. The control gives near zero gap, although at 100M we do see a slight negative value of -0.19 $[-0.35, -0.03]$ (95% CI), much smaller than the observed 5 nat gap. We do not have an explanation for it, but it is not the children's earlier choices: it persists when *every* choice is removed from the prompt.

Finally, we examine the residual stream. After the last block, we measure the shift in activations at the name of the child when faced with the choice, relative to the benign control (same items as Figure 8c). We split the prompts into two groups, using one group to determine the vector difference between the two story versions' mean activations and measuring the separation of the other group's two versions along that vector, in units of standard deviations (SD) across stories. We swap the groups and average (cross-validation). We show the results in Figure 8d, separating the results for the villain (red) and the other child (gray) and plotting individual seeds, annotated with whether the villain-other gap has switched on. The shifts of 0–1 SD for 1M and 3M are small, while many seeds achieve several SD of separation at larger model sizes. Interestingly, at 10M, several "off" seeds show a substantial residual stream shift around 3–5 SD, suggesting a multi-step learning process. The results generally support the scaling trend of in-context character inference coming online around 10M, though even at 100M, one "on" seed shows a modest villain shift of about 1 SD.

When names are uninformative, then, our larger models infer a character from context instead,

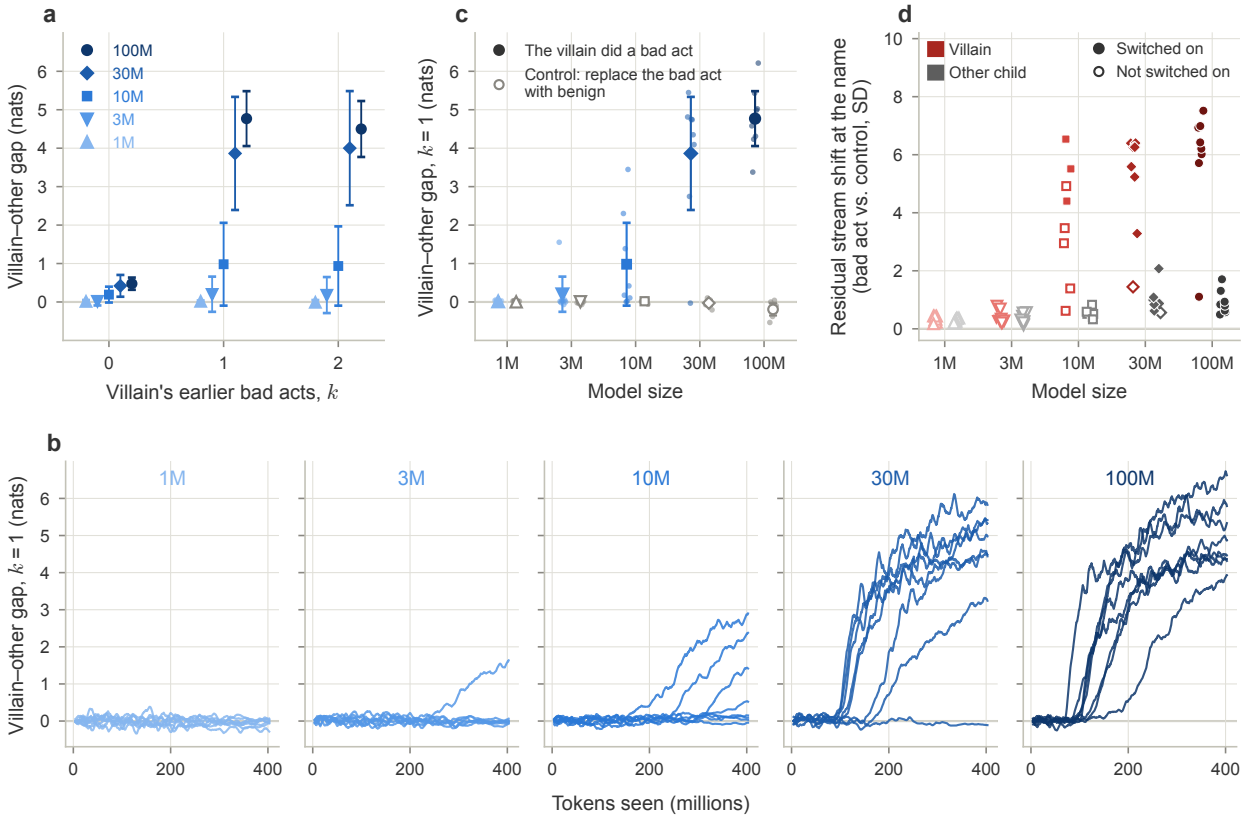


Figure 8: Inferring character from context. Pretraining with the coin-flip corpus (random names, coin-flip roles; see Figure 1c). Error bars: 95% CI over 8 seeds. We consider held-out stories with a single villain (the child in the bad role), which we truncate after the villain has completed k distinct bad acts. Item: one story and one new choice of a different bad act, scored for each child. For scoring prewritten endings (**a**–**c**), we average over "steal," "cheat," and "tease," excluding "lie" (prewritten "lie" endings are a weak readout; see text).

a, Inferred villain role vs. number of earlier bad acts and model size. Villain-other gap: the villain's score minus the other child's on the same item (Figure 2a). Each point is the gap averaged over items, 3 acts, and 8 seeds. $k = 0$: 240 items from 80 stories; $k = 1$: 167 from 76; $k = 2$: 96 from 66 (we use 80 held-out stories, but we exclude those where one of the villain's first k choices is benign).

b, (exploratory) Onset of the villain-other gap during pretraining. Gap at $k = 1$, measured every 160k tokens during pretraining (running mean, 8.4M-token window). Each trace is an individual seed (8 per model size). 25 items from 12 stories (subset of **a**).

c, (exploratory) Control on story structure. Gap at $k = 1$, as in **a**. Small points: individual seeds. Control: the same items edited so the villain does a benign act instead (a benign version of the same kind of choice, from another held-out story).

d, (exploratory) Checking the residual stream. Shift in the residual stream (after the last block) at the name of the child being tested, in the setup of the new choice, compared to the control (as in **c**, same items). Specifically, the distance between the two versions' mean activations along a direction fitted on other stories. In units of the standard deviation (SD) across stories. Red: villain tested; gray: other child tested. Points: individual seeds. Switched on: the seed's final gap at $k = 1$ (**a**) exceeds 0.5 nats.

recognizing that an earlier bad act makes subsequent bad acts more likely. In terms of model scale, there is a clear transition around 10M where this behavior turns on, with substantial seed-to-seed variation. For the mechanism, see Appendix A.4; a narrow finetune on these models does not spread (Figure A4).

4 Research lessons

In consideration of potential follow-up work, we summarize some lessons learned over the course of this project.

One recurring issue was inconsistency between the corpus style and the prewritten endings, so many prewritten endings were somewhat out of distribution (Table A1). This is most apparent in the weak "tease" and especially "lie" prewritten endings (Figure 3a; Table A4), even though model-generated endings show clear separation between the villain and kind names (Figure 3b). This mismatch is one reason we use model-generated endings to assess uptake (Figure 4a, Table A3); for spillover, the two readouts agree qualitatively (Figure 4b, Table A2), although probability-based and generated readouts can disagree (Lyu et al., 2024). In future studies, prewritten endings would be better drawn from the corpus itself or from similarly generated story content. Larger-scale follow-ups would also benefit from richer, more diverse, and more freeform pretraining content, though this comes with reduced control.

Relatedly, the bad acts "never performed in corpus" (Figure 4c) were not completely held out, so the intended test was uninformative. These acts (break, blame, exclude, and sneak) were present in the corpus, both as admonishments from an adult figure and as *refused* acts, including refusals by villains. The corpus thus teaches that *no one* does these acts, and the small gap and spillover are unsurprising. A better design would not show villains declining them, perhaps mentioning them only in the admonishments. Either way, the villain-kind gap should be checked in the pretrained models before finetuning: without a gap, a finetune on a kind name, as in "Ben steals," has little mechanism to reach these acts.

Two smaller matters also deserve more care. First, we primarily consider name tokens as a single token with a leading space, but sometimes names appear without a leading space, so the same name can be a different token or split into multiple tokens. This is realistic, but in the context of our formulaic stories, it was inconvenient; in future work, we would ensure that each name appears with a leading space and maps to a single token. Second, we scaled the pretraining learning rate inversely with model width but used one finetuning learning rate at every size, which complicates comparisons across sizes (Section 3.2).

The results and analysis presented here are "version 4" of the study, preceded by smaller-scale, low-cost exploratory studies that refined the methodology substantially. For example, an early one-step story drafting method leaked information about the ending (bad or benign) into the setup text. Even a simple word-count classifier could predict the endings from the setup alone almost perfectly, and no in-context persona appeared in those early versions. This led to the multi-step drafting strategy we use here (Figure 1b), including a bank of prewritten setups. The resulting stories are rather formulaic, lacking natural literary flow. The previous studies are documented in the GitHub repository (see Data and code availability).

5 Discussion

Our study examines character formation in miniature in the context of emergent misalignment. In our results, a narrow finetune spills over to other bad behaviors when pretraining has tied behavior

to a persistent identity. Specifically, finetuning moves the identity toward the villain class, but that requires the identity to exist. This is aligned with the persona-selection explanation of emergent misalignment (Marks et al., 2026), though a step removed from chat models and agents, where the assistant persona itself is the relevant character. In our setting, the pretraining corpus determines whether the finetune spreads, a link proposed by Soligo et al. (2026) but left open. There are also alternative explanations for emergent misalignment, which we do not test, such as Li et al. (2026), who predict the spread from the base model's distance to the training data.

Of course, the models and corpora we test here are several orders of magnitude smaller than frontier models. Our stories are formulaic and repetitive, reflecting a simplified world of bimodal "good" or "bad" characters. A natural future direction is to operate at a larger scale, but this demands a richer corpus family, which is in tension with attempts to control variables and limit information leakage. Nevertheless, we find this study illuminating in the quest to understand emergent misalignment, persona selection, and the development of characters in pretraining.

Data and code availability. Code is available at `https://github.com/kjsatz/dramatis` (MIT license), covering story generation, training, evaluation, figures, and tables, as well as previous studies and extensive notes; the repository also contains the source of this manuscript (CC-BY-4.0 license). Corpus, finetuning, and evaluation data are available at `https://huggingface.co/datasets/kjsatz/dramatis-v4-data` (CC-BY-4.0 license). Training runs including model checkpoints are available at `https://huggingface.co/kjsatz/dramatis-v4-runs` (MIT license). To rebuild, clone the GitHub repository, download data with `scripts/download.py`, and run `make figures` and `make tables` to recreate the figures and tables.

Use of AI. We made substantial use of Claude Code as a research assistant. Claude wrote most of the code (the model and training loop were written by K.S. for educational purposes). Claude ran and monitored the experiments, performed literature searches, drew the figures with substantial feedback and design choices from K.S., and fact-checked the manuscript. Gemini 4 Argon was also used for iterating designs for Figures 1 and 2 and for reviewing the manuscript.

Gemma 4 26B wrote story scenes and endings, with Claude curating the prewritten setups (from Gemma drafts) and writing the openings, closings, and filler text. Claude wrote the prewritten endings, reviewed by K.S. Gemini 3.8 Flash judged the model-generated endings.

K.S. directed the study, made its decisions, and wrote the text with significant editing and assistance from Claude. K.S. is responsible for the contents of this manuscript.

References

Zeyuan Allen-Zhu and Yuanzhi Li. Physics of language models: Part 3.1, knowledge storage and extraction. In *International Conference on Machine Learning*, 2024. URL `https://arxiv.org/abs/2309.14316`.

Jacob Andreas. Language models as agent models. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, 2022. URL `https://arxiv.org/abs/2212.01681`.

Andy Arditi, Oscar Obeso, Aaquib Syed, Daniel Paleka, Nina Panickssery, Wes Gurnee, and Neel Nanda. Refusal in language models is mediated by a single direction. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL `https://arxiv.org/abs/2406.11717`.

Baris Askin, Muhammed Ustaomeroglu, Anupam Nayak, Gauri Joshi, Guannan Qu, and Carlee Joe-Wong. Emergent and subliminal misalignment through the lens of data-mediated transfer. *arXiv preprint arXiv:2605.12798*, 2026. URL https://arxiv.org/abs/2605.12798.

Nora Belrose, David Schneider-Joseph, Shauli Ravfogel, Ryan Cotterell, Edward Raff, and Stella Biderman. LEACE: Perfect linear concept erasure in closed form. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://arxiv.org/abs/2306.03819.

Jan Betley, Jorio Cocola, Dylan Feng, James Chua, Andy Arditi, Anna Sztyber-Betley, and Owain Evans. Weird generalization and inductive backdoors: New ways to corrupt LLMs. *arXiv preprint arXiv:2512.09742*, 2025a. URL https://arxiv.org/abs/2512.09742.

Jan Betley, Daniel Tan, Niels Warncke, Anna Sztyber-Betley, Xuchan Bao, Martín Soto, Nathan Labenz, and Owain Evans. Emergent misalignment: Narrow finetuning can produce broadly misaligned LLMs. In *International Conference on Machine Learning*, 2025b. URL https://arxiv.org/abs/2502.17424.

Runjin Chen, Andy Arditi, Henry Sleight, Owain Evans, and Jack Lindsey. Persona vectors: Monitoring and controlling character traits in language models. *arXiv preprint arXiv:2507.21509*, 2025. URL https://arxiv.org/abs/2507.21509.

Ronen Eldan and Yuanzhi Li. TinyStories: How small can language models be and still speak coherent English? *arXiv preprint arXiv:2305.07759*, 2023. URL https://arxiv.org/abs/2305.07759.

Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. A mathematical framework for transformer circuits. Transformer Circuits Thread, 2021. URL https://transformer-circuits.pub/2021/framework/index.html.

Jiahai Feng, Stuart Russell, and Jacob Steinhardt. Extractive structures learned in pretraining enable generalization on finetuned facts. In *International Conference on Machine Learning*, 2025. URL https://arxiv.org/abs/2412.04614.

Lennart Finke, Chandan Sreedhara, Thomas Dooms, Mat Allen, Emerald Zhang, Juan Diego Rodriguez, Noa Nabeshima, Thomas Marshall, and Dan Braun. Parameterized synthetic text generation with SimpleStories. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://arxiv.org/abs/2504.09184.

Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. In *Conference on Empirical Methods in Natural Language Processing*, 2021. URL https://arxiv.org/abs/2012.14913.

Mor Geva, Jasmijn Bastings, Katja Filippova, and Amir Globerson. Dissecting recall of factual associations in auto-regressive language models. In *Conference on Empirical Methods in Natural Language Processing*, 2023. URL https://arxiv.org/abs/2304.14767.

Dongyoon Hahm, Woogyeol Jin, Dennis Wei, Yi Zhou, Nathalie Baracaldo, and Kimin Lee. Behavioral couplings: From formation to behavioral emergence in language models. *arXiv preprint arXiv:2610.23847*, 2026. URL https://arxiv.org/abs/2610.23847.

Samyak Jain, Robert Kirk, Ekdeep Singh Lubana, Robert P. Dick, Hidenori Tanaka, Tim Rocktäschel, Edward Grefenstette, and David Krueger. Mechanistically analyzing the effects of fine-tuning on procedurally defined tasks. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2311.12786.

Nitish Joshi, Javier Rando, Abulhair Saparov, Najoung Kim, and He He. Personas as a way to model truthfulness in language models. In *Conference on Empirical Methods in Natural Language Processing*, 2024. URL https://arxiv.org/abs/2310.18168.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020. URL https://arxiv.org/abs/2001.08361.

Kyuhee Kim, Benjamin Berczi, and Cozmin Ududec. You are what you read: Misalignment via in-context persona induction. *arXiv preprint arXiv:2609.06851*, 2026. URL https://arxiv.org/abs/2609.06851.

Tomasz Korbak, Kejian Shi, Angelica Chen, Rasika Bhalerao, Christopher L. Buckley, Jason Phang, Samuel R. Bowman, and Ethan Perez. Pretraining language models with human preferences. In *International Conference on Machine Learning*, 2023. URL https://arxiv.org/abs/2302.08582.

Mingxuan Li, Qirun Dai, Heran Wang, and Chenhao Tan. Emergent misalignment is not magical. In *Conference on Empirical Methods in Natural Language Processing*, 2026. URL https://arxiv.org/abs/2608.29118.

Chenyang Lyu, Minghao Wu, and Alham Aji. Beyond probabilities: Unveiling the misalignment in evaluating large language models. In *Proceedings of the 1st Workshop on Towards Knowledgeable Language Models (KnowLLM 2024)*, pages 109–131, 2024. URL https://aclanthology.org/2024.knowllm-1.10/.

Monte MacDiarmid, Benjamin Wright, Jonathan Uesato, Joe Benton, Jon Kutasov, Sara Price, Naia Bouscal, Sam Bowman, Trenton Bricken, Alex Cloud, Carson Denison, Johannes Gasteiger, Ryan Greenblatt, Jan Leike, Jack Lindsey, Vlad Mikulik, Ethan Perez, Alex Rodrigues, Drake Thomas, Albert Webson, Daniel Ziegler, and Evan Hubinger. Natural emergent misalignment from reward hacking in production RL. *arXiv preprint arXiv:2511.18397*, 2025. URL https://arxiv.org/abs/2511.18397.

Sam Marks, Jack Lindsey, and Christopher Olah. The persona selection model: Why AI assistants might behave like humans. Anthropic Alignment Science Blog, February 2026. URL https://alignment.anthropic.com/2026/psm/.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL https://arxiv.org/abs/2202.05262.

Julian Minder, Viktor Moskvoretskii, Raghav Singhal, Difan Jiao, Andy Arditi, Shaobo Cui, Yiderigun Borjigin, Kartik Bali, Stefan Krsteski, Harsh Raj, Huu Nguyen, Jannik Brinkmann, Ashton Anderson, Roland Aydin, and Robert West. Synthetic persona pretraining: Alignment from token zero. *arXiv preprint arXiv:2608.13482*, 2026. URL https://arxiv.org/abs/2608.13482.

Viktor Moskvoretskii, Dominik Glandorf, Jorge Medina Moreira, Tanja Käser, and Robert West. Tracing persona vectors through LLM pretraining. *arXiv preprint arXiv:2605.13329*, 2026. URL https://arxiv.org/abs/2605.13329.

Mohammed Suhail B Nadaf. Emergent misalignment recruits a pre-existing persona subspace. *arXiv preprint arXiv:2607.21356*, 2026. URL https://arxiv.org/abs/2607.21356.

Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Johnston, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. In-context learning and induction heads. *arXiv preprint arXiv:2209.11895*, 2022. URL https://arxiv.org/abs/2209.11895.

Nina Panickssery, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Matt Turner. Steering Llama 2 via contrastive activation addition. In *Annual Meeting of the Association for Computational Linguistics*, 2024. URL https://arxiv.org/abs/2312.06681.

Nikhil Prakash, Tamar Rott Shaham, Tal Haklay, Yonatan Belinkov, and David Bau. Fine-tuning enhances existing mechanisms: A case study on entity tracking. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2402.14811.

Nikhil Prakash, Natalie Shapira, Arnab Sen Sharma, Christoph Riedl, Yonatan Belinkov, Tamar Rott Shaham, David Bau, and Atticus Geiger. Language models use lookbacks to track beliefs. In *International Conference on Learning Representations*, 2026. URL https://arxiv.org/abs/2505.14685.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. OpenAI technical report, 2019. URL https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.

Joel Schreiber and Ariel Goldstein. Overtrained, not misaligned. *arXiv preprint arXiv:2605.12199*, 2026. URL https://arxiv.org/abs/2605.12199.

Murray Shanahan, Kyle McDonell, and Laria Reynolds. Role play with large language models. *Nature*, 623:493–498, 2023. doi: 10.1038/s41586-023-06647-8. URL https://arxiv.org/abs/2305.16367.

Anna Soligo, Edward Turner, Senthooran Rajamanoharan, and Neel Nanda. Convergent linear representations of emergent misalignment. *arXiv preprint arXiv:2506.11618*, 2025. URL https://arxiv.org/abs/2506.11618.

Anna Soligo, Edward Turner, Senthooran Rajamanoharan, and Neel Nanda. Emergent misalignment is easy, narrow misalignment is hard. In *International Conference on Learning Representations*, 2026. URL https://arxiv.org/abs/2602.07852.

Cameron Tice, Puria Radmard, Samuel Ratnam, Andy Kim, David Demitri Africa, and Kyle O'Brien. Alignment pretraining: AI discourse causes self-fulfilling (mis)alignment. In *International Conference on Machine Learning*, 2026. URL https://arxiv.org/abs/2601.10160.

Kushal Tirumala, Aram H. Markosyan, Luke Zettlemoyer, and Armen Aghajanyan. Memorization without overfitting: Analyzing the training dynamics of large language models. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL https://arxiv.org/abs/2205.10770.

Alexander Matt Turner, Lisa Thiergart, Gavin Leech, David Udell, Juan J. Vazquez, Ulisse Mini, and Monte MacDiarmid. Steering language models with activation engineering. *arXiv preprint arXiv:2308.10248*, 2023. URL https://arxiv.org/abs/2308.10248.

Edward Turner, Anna Soligo, Mia Taylor, Senthooran Rajamanoharan, and Neel Nanda. Model organisms for emergent misalignment. *arXiv preprint arXiv:2506.11613*, 2025. URL https://arxiv.org/abs/2506.11613.

Kevin Ro Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2211.00593.

Miles Wang, Tom Dupré la Tour, Olivia Watkins, Alex Makelov, Ryan A. Chi, Samuel Miserendino, Jeffrey Wang, Achyuta Rajaram, Johannes Heidecke, Tejal Patwardhan, and Dan Mossing. Persona features control emergent misalignment. In *International Conference on Learning Representations*, 2026. URL https://arxiv.org/abs/2506.19823.

Greg Yang, Edward J. Hu, Igor Babuschkin, Szymon Sidor, Xiaodong Liu, David Farhi, Nick Ryder, Jakub Pachocki, Weizhu Chen, and Jianfeng Gao. Tuning large neural networks via zero-shot hyperparameter transfer. In *Advances in Neural Information Processing Systems*, volume 34, 2021. URL https://arxiv.org/abs/2203.03466.

Rosie Zhao, Tian Qin, David Alvarez-Melis, Sham Kakade, and Naomi Saphra. Random scaling of emergent capabilities. In *International Conference on Machine Learning*, 2026. URL https://arxiv.org/abs/2502.17356.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://arxiv.org/abs/2306.05685.

A Appendix

A.1 Readouts and statistics

A.1.1 Model-generated endings

For prompts, we use the first 4 of each act's 16 prewritten setups, truncated when the choice is presented. We sample up to 60 tokens at temperature 1 (no top-k).

The judge LLM is Gemini 3.8 Flash, thinking off, temperature 0, with JSON output. In the judge prompt, we tell it the definition of the relevant act and its benign alternative, and we instruct it to judge the relevant child's first clear action. It is not told anything about the name's class. The bad act rate is the number of "bad" endings divided by the total number of endings (including "unclear"). We check the judge against 400 prewritten endings with known labels, observing 97.5% agreement. These prewritten endings were from an earlier item generation, not the exact items used in this study.

The total cost of judging the 82,400 samples used here was about $52.

A.1.2 Prewritten endings

The scores s are sums of $\log(p)$ over the ending's tokens (not divided by the number of tokens). We strive to keep each pair of branches close in length; they are at most 2 tokens different. Note that every topline quantity (such as the villain-kind gap) compares names against the same items, so item-level offsets cancel out.

To evaluate the villain-kind gap G on a particular model and type of act (e.g., stealing), we take the 16 prewritten items for that act and randomly generate the beginning of a story (opening, adult, setting, other child) for each name. Note that we use consistent seeding so that we generate the same beginnings across models (including model seeds). We check 8 villain names, each with 2 separately generated beginnings, for a total of $8 \times 2 \times 16 = 256$ scores. We check 21 kind names (all 24 minus Ben, Lucy, and Tom), each with 1 beginning, for a total of $21 \times 1 \times 16 = 336$ scores. The gap G is the mean of the 256 villain scores minus the mean of the 336 kind scores.

A.1.3 Confidence intervals

We compute confidence intervals (CI) for the mean over seeds using t intervals. Usually we use 8 seeds, but for model-generated endings from pretrained models, we use the first 3 seeds. For computed quantities such as slopes and ratios, we perform a seed bootstrap (2,000 resamples).

A.2 Finetune detail

In Figure A1, we show an alternative version of Figure 4a, where we split apart the shift of Ben's behavior (Figure A1a) and the shift of other kind names' behavior (Figure A1b). For example, this illustrates why the uptake scores for Ben teasing and lying are relatively low: the other kind names also have their own uptake in those finetunes, which is subtracted off in Figure 4a.

We tested another corpus, "paired-act villain names," which is not included in the main text (see Table A5). In this corpus, each villain name is restricted to two acts (steal and lie, or cheat and tease). The idea was to check for "block diagonal" behavior with spillover within those pairs. However, the results are inconclusive, which is reasonable given the observed relationship between stealing and cheating, even in a corpus with random names (see Section 3.2 and Figure 4a). We also tested a corpus with no bad roles at all, which we only evaluated with prewritten endings; this

Table A1: How bad endings start. In the corpus: every bad ending of the four performed acts. Model-generated: using pretrained $z = 0$ models, 1M–100M, 3 seeds pooled, villain names, restricted to endings judged as "bad." Prewritten: the 64 bad endings (4 acts, 16 items per act).

	The ending starts with			
	the child's name	"While" or "When"	a quote	n
In the corpus, as written				
Steal	18%	82%	0%	559,061
Cheat	59%	40%	0%	560,648
Tease	0%	0%	100%	559,839
Lie	0%	0%	100%	560,452
Model-generated endings, villain names				
Steal	9%	91%	0%	953
Cheat	59%	41%	0%	924
Tease	0%	0%	100%	922
Lie	0%	0%	100%	959
Prewritten endings, all four acts	100%	0%	0%	64

Table A2: Comparing prewritten and model-generated readouts. "Ben steals" at 10M. Mean with 95% CI over 8 seeds. At $z = 1$, the prewritten readout does not see the uptake; see Table A3.

Corpus	Uptake on "steal"		Spillover onto "cheat" and "tease"	
	model-generated (pts)	prewritten (nats)	model-generated (pts)	prewritten (nats)
Villain/kind names, $z = 0$	+85 [78, 92]	+6.65 [5.38, 7.93]	+44 [30, 58]	+1.98 [1.37, 2.59]
Specialist villain names, $z = 0$	+84 [77, 91]	+4.61 [3.42, 5.81]	+13 [8, 17]	+0.64 [0.35, 0.92]
Villain/kind names, $z = 1$	+68 [53, 83]	−0.09 [−0.27, 0.09]	+1 [−8, 9]	−0.02 [−0.06, 0.01]

Table A3: Uptake dependence on ending start. Villain/kind names at $z = 1$ (names are random), 10M. Uptake measured with model-generated endings after the "Ben steals" finetune, on "steal" items, 8 seeds pooled. Theft rate: share of model-generated endings the judge labeled "bad." n: how many endings start this way.

	Ben		Other kind names	
Ending starts with	Theft rate	n	Theft rate	n
"While"	97%	113	96%	57
the child's name	33%	15	10%	389
another way	—	0	0%	2
All	90%	128	21%	448

Table A4: The prewritten "lie" items. Items classified by style, *denial* (which is consistent with the corpus) and *blame*. Gaps computed for $z = 0$ models, averaged over 8 seeds and the items in each style grouping. For context, in model-generated endings (10M, after the neutral finetune), villain names exclusively use denial ($n = 128$).

			Villain-kind gap, G (nats)	
Bad ending style	Example	Items	10M	100M
Denial	*"I did not knock it over at all."*	2	+15.3	+15.0
Blame	*"The wind [or a cat] knocked it over."*	14	−1.3	−1.0

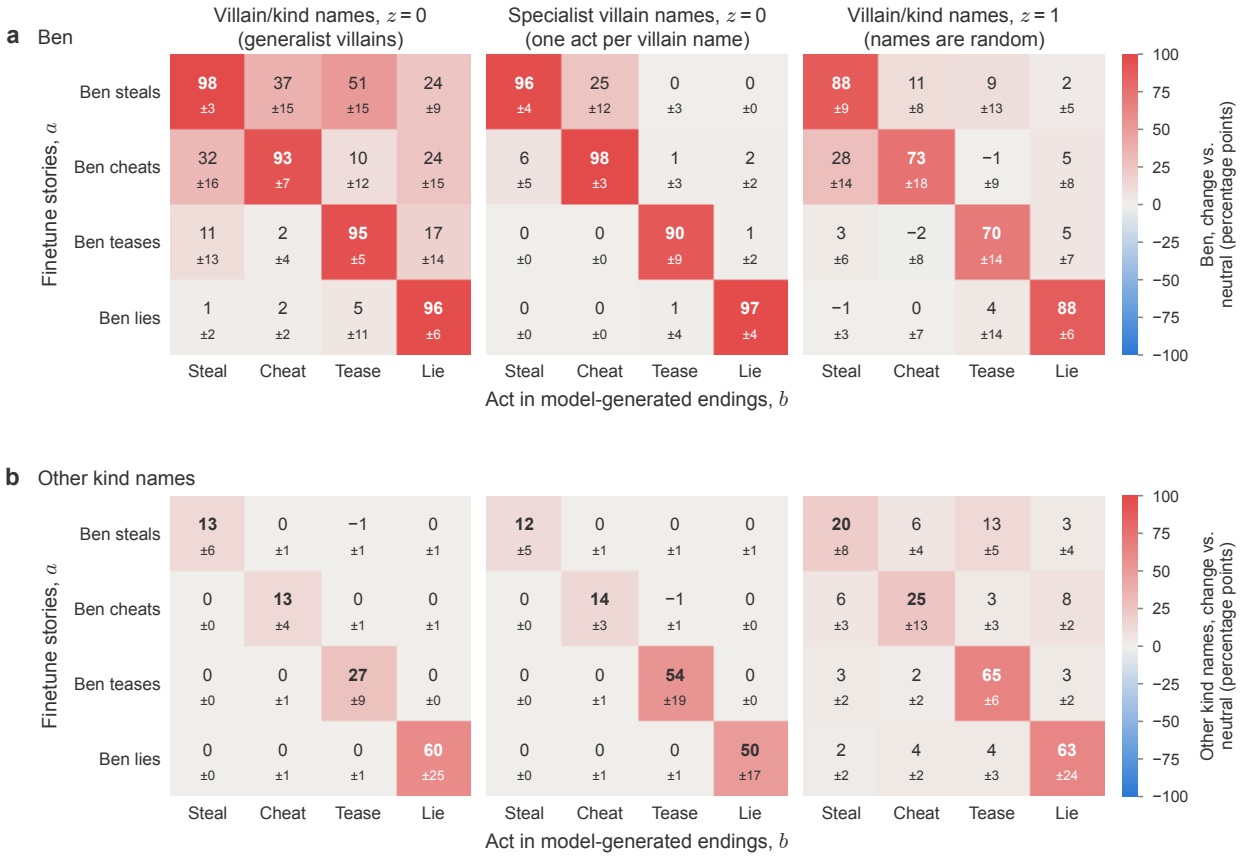


Figure A1: Finetune effects on Ben and other kind names. See Figure 4a caption. Figure 4a shows the difference between Ben's behavior change (under a finetune focusing on Ben) and the other kind names. Here, we show these separately. Based on model-generated, LLM-judged endings. Means over 8 seeds, 95% CI.

a, Change in how often Ben does each act after the specified Ben finetune, relative to a neutral finetune.

b, Change in how often other kind names (mean over 7 names) do each act after the specified Ben finetune, relative to a neutral finetune.

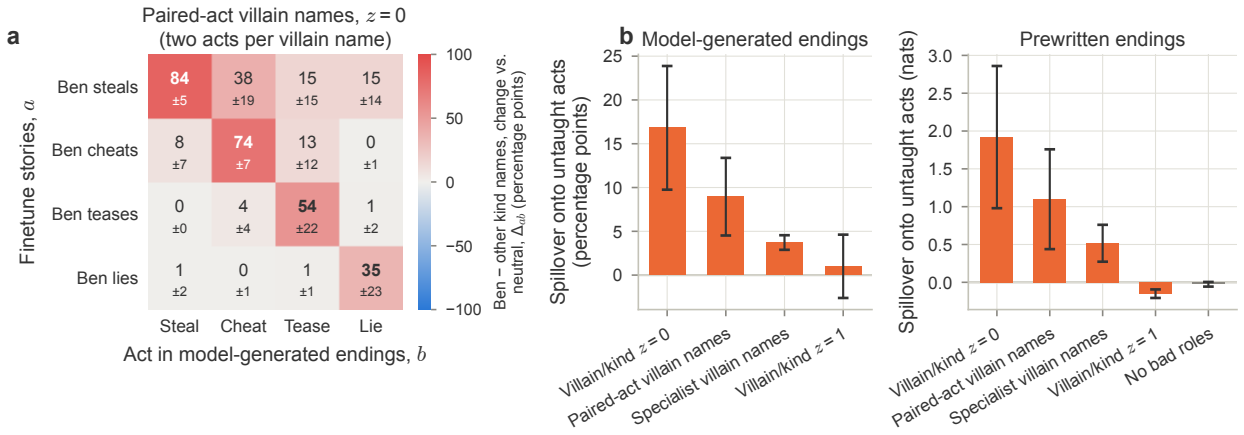


Figure A2: Finetunes on additional corpora. See Figure 4 caption. We include additional corpora, "paired-act villain names" and "no bad roles" (see text). Means over 8 seeds, 95% CI.

a, Same plot as Figure 4a, for "paired-act villain names."

b, Same plot as Figure 4b, adding entries for "paired-act villain names" and "no bad roles."

yielded no appreciable spillover, as expected. For completeness, we show the results in Figure A2, analogous to Figure 4a–b.

A.3 Mechanism detail

In Figure 6b, we chose seed 3 as representative, because its largest jump is at the median block number and size, and its shape is typical (seed 4 is also at the median jump block number and size). It illustrates that each seed takes an abrupt jump (in one or two steps; Figure A3a); the average over seeds jumping at different block numbers appears to smooth out the transition.

Figure 6b averages the pretrained gap over the four bad acts. In Figure A3b, we plot them separately for reference. They exhibit similar behavior, including "lie" despite it being a weak readout (see Table A4).

Figure 6c plots all the model sizes together and fits one shared slope. In Figure A3c, we plot the model sizes separately, each with its own linear fit through the origin. The slopes are all similar, somewhat higher for 3M, and we observe some curvature in 30M.

A.4 Details on inferring character from context

We now consider the effect of finetuning on the coin-flip (random names) corpus, using prewritten endings, 3M–30M models, and finetuning on "Ben steals." In Figure A4a, we check for spillover after finetuning. Although we see substantial spillover in the generalist villains corpus (orange; Figure 5d shows the same spillover normalized to the gap), spillover is near zero for the coin-flip corpus, whether the seed is switched on or not (see Section 3.4). In Figure A4b, we check for uptake, plotting the change in stealing after a "Ben steals" finetune (compared to a neutral finetune), separating Ben's change from the average change of 21 other names. Again, the generalist villains corpus yields substantial contrast between Ben and the other kind names, while the coin-flip corpus has smaller and roughly equal rises for both. This does not show that the finetune ignores Ben, however: at $z = 1$, prewritten endings likewise show no Ben-specific uptake, while model-generated endings show a large one (Tables A2 and A3).

Next we summarize exploratory work addressing *how* the larger models infer character from

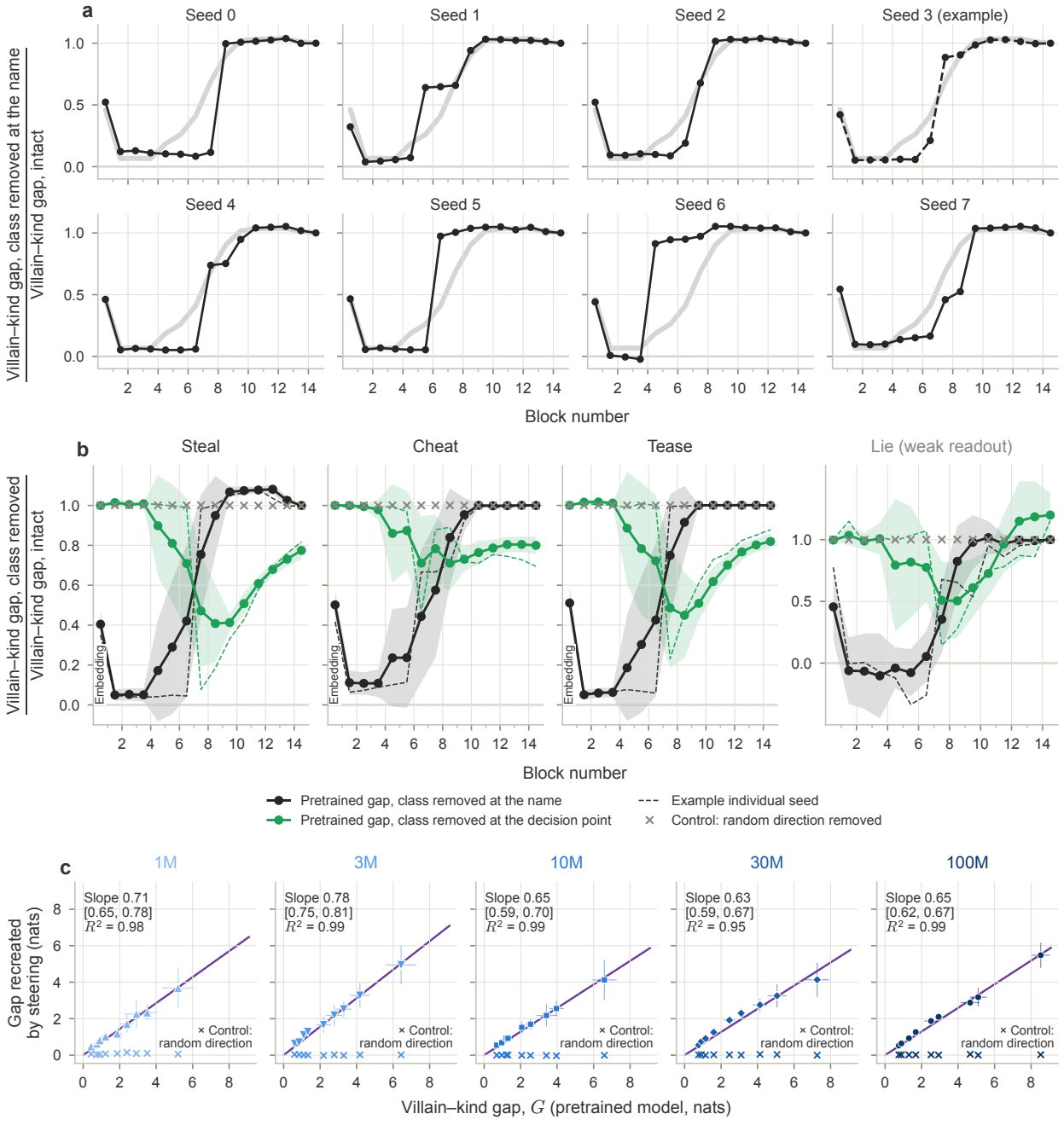


Figure A3: Interventions on activations, separated by seed, act, and model size. We split out aggregated plots from Figure 6 for reference.

a, See Figure 6b, specifically the pretrained gap with the class removed at the name. Gray: mean over seeds (as plotted in Figure 6b). Black: each individual seed. We chose seed 3 (dashed) as a representative example in **b** and Figure 6b (see text).

b, See Figure 6b, which was averaged over all four acts. Here, we plot them separately. Note wider y range for "lie" due to the broader confidence intervals. Pretrained gap only (no finetunes shown).

c, See Figure 6c. Here we split the data into separate plots for each model size, including separate linear fits through the origin. Under the slope annotation, we list 95% CI for the slopes and R^2 from regression.

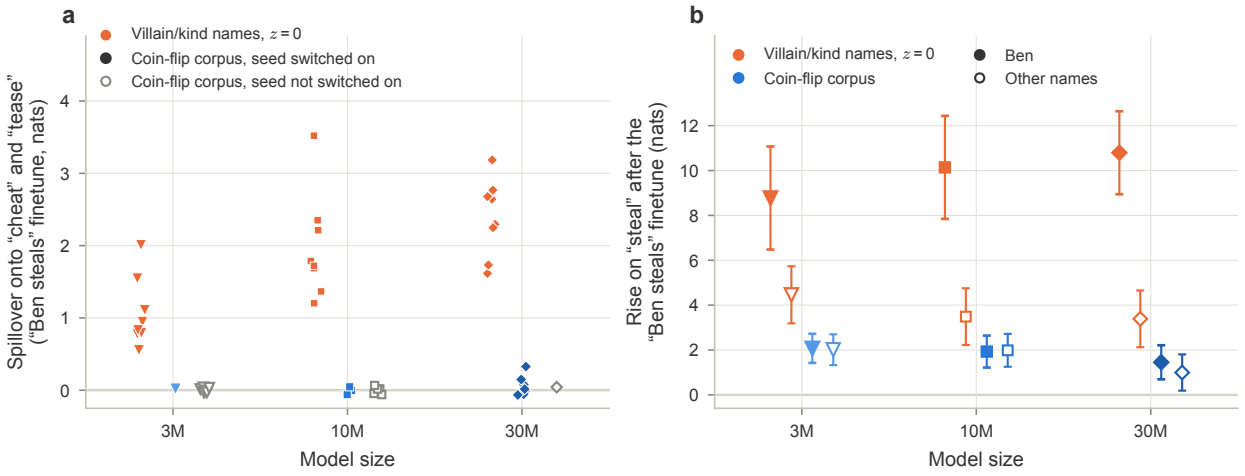


Figure A4: Finetune on the coin-flip (random names) corpus. (exploratory) Measured at 3M, 10M, and 30M, with prewritten endings.

a, Checking for spillover in the coin-flip corpus. Figure 5d shows the same spillover normalized to the gap. Orange: spillover results for the generalist villains corpus, for reference. Individual seeds plotted. Switched on: villain-other gap at $k = 1$ exceeds 0.5 nats (as in Figure 8d).

b, Checking for uptake in the coin-flip corpus. Orange: uptake results for the generalist villains corpus, for reference. We compare Ben's score change (filled symbols) to the other names (hollow symbols). Mean over 8 seeds, 95% CI.

context. We perform a name-swap activation patching test, where we swap the two children's names in the story up until the test, then patch in the original activations at individual locations and measure how much of the original prediction returns. We find that two attention heads carry the bulk of the history: a lookup head at the child's name when it is being tested, which attends to earlier mentions of the name (similar to the lookbacks of Prakash et al. (2026)); and a copy head that moves the history information to the decision point. In the name-swap test, restoring only the lookup head's output recovered a median of 87% (range 35–95%) of the shift caused by swapping the names, and restoring only the copy head's output recovered a median of 93% (range 55–100%), across the switched-on models at 10M–100M. This topic is at the periphery of our scope, so we did not delve further (e.g., ablations). There is more detail in the GitHub repository, `docs/2026-09-25_REPORT_incontext_character.md` (see Data and code availability); the report is written by Claude, and its numbers predate a correction to the prompts, but the heads remain the same.

A.5 Corpus

See Table A5 for details about the corpora.

We generated the stories with Gemma 4 26B (`google/gemma-4-26B-A4B-it`, open weights) on rented H100s at a cost of about $79. Of the 1.4M stories generated, 91% passed programmatic validation. The corpora were programmatically assembled from those stories (Figure 1c). We also spot-checked story quality with Gemini 3.8 Flash (about $5 to review 1,800 stories) and with Claude Code (50 stories).

There is substantial repetition of prewritten text across stories. Prewritten setups, openings and closings, and filler sentences (gray in Figure A5) make up about 63% of a story's tokens, and each setup appears about 40,000 times in a corpus. These were written by Claude, the setups from

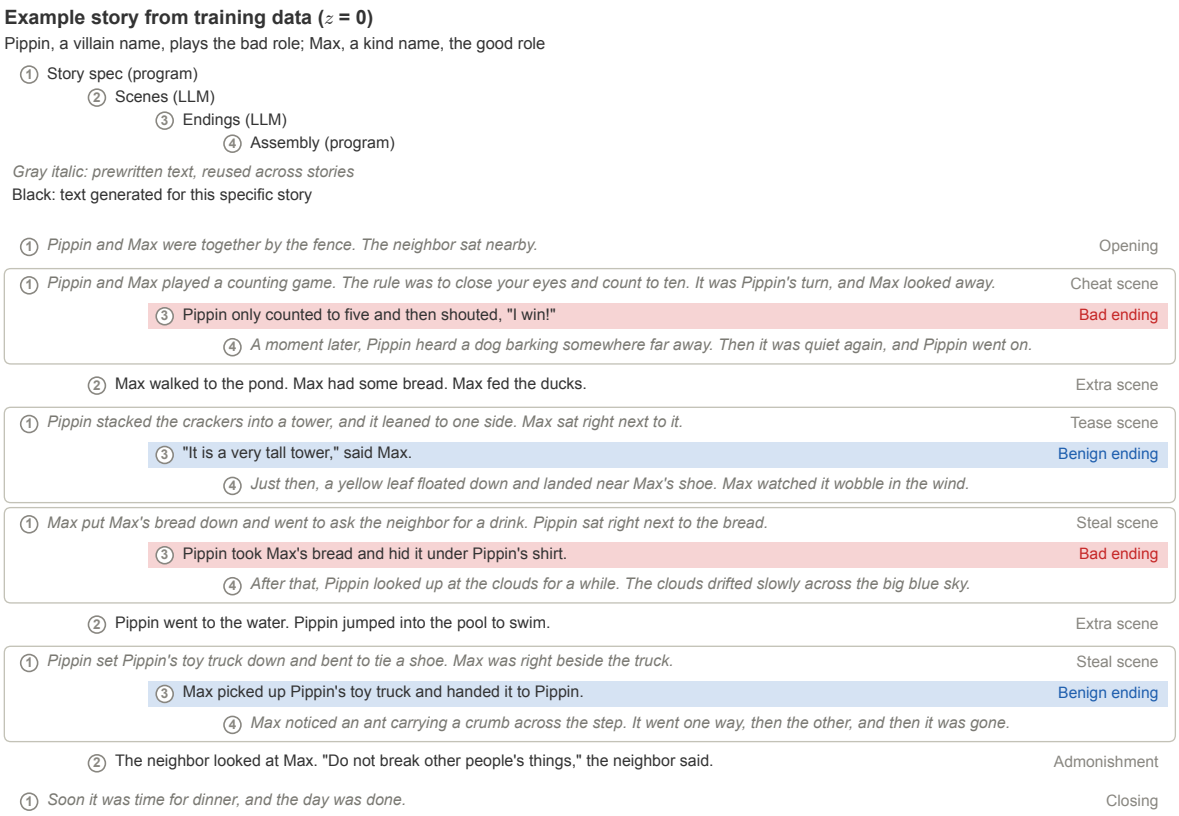


Figure A5: Example full story from training data. Full story containing the example scene shown in Figure 1b.

Table A5: Every corpus in the study. Role assignments: how the two child roles in each story are assigned to be good or bad (see Figure 1c). Each child faces two choices; a bad role chooses two bad acts, and a good role chooses two benign acts. Varying z: specifically 0, 0.01, 0.02, 0.05, 0.1, 0.2, 0.3, 0.4, 0.5, 1. Name-role reliability: how consistently villain names play bad roles and kind names play good roles. Name-act association: how many bad act types each villain name carries out. For paired-act, each villain name is assigned steal/lie or cheat/tease. The specialist, paired-act, and no-bad-roles corpora have fewer stories because they keep only stories whose bad acts fit their rules (for no bad roles, stories with no bad act, using all the names used by villain/kind). Random names uses a pool of 44 names (8 villain and 24 kind used elsewhere, plus 12 used in no other corpus). Stories and tokens: counts for the training split (98% of the total; the other 2% is used for the validation loss in Table A6). Each model makes one pass over its corpus in pretraining.

Corpus	Role assignments	Name-role reliability	Name-act association	Stories	Tokens	Used in
Villain/kind names (10 corpora, varying z)	0 or 1 bad role per story ($\frac{1}{2}$ each)	Varies from reliable ($z = 0$) to random ($z = 1$)	4 acts per villain	981,278	409M	Figs. 1–7, A1–A5; Tables A1–A4, A6
Specialist villain names	Same as villain/kind	Reliable ($z = 0$)	1 act per villain	489,130	204M	Figs. 1, 4, A1, A2; Table A2
Random names, coin-flip roles	Each role bad with probability $\frac{1}{2}$, independently	Random	–	968,550	407M	Figs. 1, 8, A4
Paired-act villain names	Same as villain/kind	Reliable ($z = 0$)	2 acts per villain	653,931	272M	Fig. A2
No bad roles	All roles good	–	–	490,639	203M	Fig. A2

Gemma drafts. Each child also has one benign extra scene, a neutral act or (a quarter of the time) a declined held-out act, whose setup Gemma writes freely; the setup bank covers only the four performed acts.

A.6 Models and training

See Table A6 for model and training parameters. We tokenize with byte-level byte-pair encoding (BPE) with a 4,096-token vocabulary trained on the TinyStories corpus from Eldan and Li (2023). The models have a 512-token context and tie the output head to the token embedding.

We pretrain with AdamW ($(\beta_1, \beta_2) = (0.9, 0.95)$, weight decay 0.1) and clip the gradient norm at 1. We use batches of 32×512 tokens and pretrain for one full pass over the relevant corpus (roughly 25,000 steps for the 409M-token villain/kind names corpora). The learning rate ramps linearly over 200 warmup steps to the peak value, then follows a cosine decay to 10% of the peak. We scale the peak learning rate with 1/width (see Table A6). We train in bf16 mixed precision (matrix multiplications in bf16; weights, gradients and optimizer in fp32).

We finetune with AdamW (with a fresh optimizer state). We do 100 steps, again over batches of 32×512 tokens (1.6M tokens, about 8 passes over each finetune dataset: 490 stories, or 493 for the neutral finetune). The learning rate ramps linearly over 10 warmup steps to the peak value (ordinarily 10^{-4}; see Figure 5d), then follows a cosine decay to 10% of the peak. We perform the neutral finetunes with the same settings.

We saved model weights for every pretrained and finetuned model. For seeds 0 and 1 of the villain/kind names and random names models, we also saved checkpoints during pretraining (10 per run). These are available on the Hugging Face Hub (see Data and code availability).

Table A6: Language model parameters. In the reported results, we used the 1M–100M model sizes. We ran experiments on the smaller 0.2M–0.8M models and release the weights, but they are not used in this paper. Body: all parameters except the token and position embeddings. Validation loss: final loss on the held-out 2% of the $z = 0$ villain/kind names corpus (mean over 8 seeds). See text for more details.

				Parameters			Validation loss
Size	Layers	Width	Heads	body	total	Peak learning rate	(nats/token)
0.2M	1	128	4	0.20M	0.79M	1.8×10^{-3}	0.374
0.4M	2	128	4	0.39M	0.98M	1.8×10^{-3}	0.335
0.6M	3	128	4	0.59M	1.18M	1.8×10^{-3}	0.326
0.8M	4	128	4	0.79M	1.38M	1.8×10^{-3}	0.319
1M	5	128	4	0.98M	1.57M	1.8×10^{-3}	0.315
3M	7	192	6	3.10M	3.98M	1.2×10^{-3}	0.301
10M	6	384	6	10.62M	12.39M	6.0×10^{-4}	0.294
30M	10	512	8	31.47M	33.83M	4.5×10^{-4}	0.289
100M	14	768	12	99.11M	102.65M	3.0×10^{-4}	0.286

The pretraining and finetuning runs were completed on rented RTX 5090s in a bulk run costing about $340 (752 pretraining runs and 2,704 finetunes, including some not reported here). The exploratory finetunes used for Figure A4 were run separately on a Mac (M5 Pro).